Authentication through Keycloak

English version of ../KEYCLOAK.md. The system has two authentication modes, switched by the AUTH_MODE variable:

Mode	What it verifies	When it is used
jwt (default)	an HS256 token signed with SECRET_KEY	local runs, the demonstration, both cloud contours
keycloak	a Keycloak RS256 token via JWKS	the customer's closed contour, corporate SSO

Both modes return a local User record: it is needed for database relationships (comment author, responsible KAM) and for RBAC. Only the way identity is confirmed differs.

Why JWT remains the default mode

The specification names Keycloak as an authentication option. For the prototype JWT needs no separate service, so the demo contour runs without Keycloak. The path to corporate SSO is nevertheless fully implemented and enabled with one variable — no code rewrite is needed if the customer requires it.

Composition

File	Purpose
keycloak/realm-export.json	The rtk-crm realm, the client, roles, demo users
backend/auth/keycloak.py	Token verification via JWKS, role parsing
backend/auth/security.py	Mode selection, RBAC by token roles
tests/test_keycloak_auth.py	20 tests: signature, claims, RBAC, the key cache

Realm structure

The rtk-crm realm contains three roles and a public client with PKCE:

- admin — administrator: reference data, users, settings;
- manager — manager: reference data and reports on all universities;
- kam — KAM: requests and reports on their own universities.

The rtk-crm-frontend client is public (no client secret), Authorization Code Flow with pkce.code.challenge.method=S256. PKCE is mandatory for a public client: without it an intercepted authorization code could be exchanged for a token.

Roles are placed into the token by two mappers: realm roles (realm_access.roles) and audience (aud). The second is needed because by default Keycloak does not set the client's aud, and without it the audience check fails.

Redirect addresses are given as patterns: http://localhost:3000/*, http://localhost:5173/* for a local run and http://* for the stand. The last pattern is required because the VM address is unknown

in advance, and Keycloak accepts `*` only at the end of the string: an entry like `http://*/*` matches no address at all and sign-in fails with `Invalid parameter: redirect_uri`.

How token verification works

```
A request with a Bearer token
        │
        ├─ AUTH_MODE=jwt ──────► HS256 with SECRET_KEY, sub = the user id
        │
        └─ AUTH_MODE=keycloak ─► RS256 with the realm public key
                                 │
                                 ├─ kid from the token header
                                 ├─ the key from the JWKS cache (TTL 300 s)
                                 ├─ checking iss / aud / exp
                                 └─ email or preferred_username
                                        │
                                        ├─ record found → sign-in
                                        └─ no record → created automatically
```

Implementation notes:

- **Key cache.** Keycloak is queried not on every HTTP request but once per `KEYCLOAK_JWKS_TTL_SECONDS`. If an unknown `kid` appears in a token, the cache is dropped and the keys are requested again — that is how rotation is handled.
- **A Keycloak failure is a 503, not a 401.** If JWKS is unreachable, the response is `503 Service Unavailable`: the token may be valid, and the client must not be logged out because of a network error.
- **Roles from the token are the source of truth.** In keycloak mode RBAC relies on `realm_access.roles` and `resource_access.<client>.roles`, not on the role in the database. A role change in Keycloak takes effect immediately, without editing the account in the CRM.
- **Account auto-creation.** If a user exists in Keycloak but not yet in the CRM, the record is created on the first sign-in with a role by priority admin > manager > kam. Local password sign-in is impossible for such accounts: `hashed_password` holds the marker `!keycloak`, which is not a valid bcrypt hash.

Running locally

```
export KEYCLOAK_ADMIN_PASSWORD=<realm-admin-password>
export DATABASE_URL=sqlite+aiosqlite:///./dev.db
export MINIO_ROOT_PASSWORD=<minio-password>
docker compose up -d keycloak

# Verify that the realm was imported
curl -s http://localhost:8080/realms/rtk-crm/.well-known/openid-configuration | head
```

The realm is imported from `keycloak/realm-export.json` on the first container start (`--import-realm`). The administration console is at http://localhost:8080.

Demo users from the realm file:

Login	Password	Role
`admin@rtk.ru`	`Admin123!`	admin
`manager@rtk.ru`	`Manager123!`	manager
`kam@rtk.ru`	`Kam123!`	kam

The passwords in the realm file are for a local demonstration. For a closed contour they must be replaced when the realm is imported.

Enabling it in the application

```
AUTH_MODE=keycloak
# The internal address: the backend fetches the public keys (JWKS) from it.
KEYCLOAK_URL=https://keycloak.example.ru
# The public address: the browser opens Keycloak at it. This is the one that ends up in the
# `iss` claim, so the token issuer is verified against it. In Docker this is
# http://keycloak:8080 for KEYCLOAK_URL and the VM's external address for this parameter.
KEYCLOAK_PUBLIC_URL=https://keycloak.example.ru
KEYCLOAK_REALM=rtk-crm
KEYCLOAK_CLIENT_ID=rtk-crm-frontend
# The token audience; an empty value disables the aud check
KEYCLOAK_AUDIENCE=rtk-crm-frontend
KEYCLOAK_JWKS_TTL_SECONDS=300
```

The frontend is switched by the same mode at build time: `VITE_AUTH_MODE=keycloak` (in Docker Compose the value is taken from `AUTH_MODE`). In that mode Keycloak performs the sign-in with Authorization Code Flow with PKCE, and the access token is inserted into requests and refreshed before they are sent. With `VITE_AUTH_MODE=jwt` the local e-mail/password form remains.

The Keycloak address for the browser is set through `KEYCLOAK_PUBLIC_URL`: `VITE_KEYCLOAK_URL` is built from it, so on the VM it is enough to set `KEYCLOAK_PUBLIC_URL=http://<vm-ip>:8080` and sign-in works without rebuilding the image.

The sign-in page

In `keycloak` mode the sign-in page offers both options: the **Sign in with Keycloak** button and the e-mail/password form below the separator. The first leads to single sign-on, the second remains a working fallback.

Before initialising keycloak-js the frontend checks the availability of Keycloak through `.well-known/openid-configuration`. This is needed because with an unreachable server keycloak-js in `check-sso` mode navigates the top window to Keycloak and leaves the user on the browser error page. The precheck prevents that: if the server does not answer, the page shows the sign-in form immediately.

Obtaining a token

The password grant (convenient for debugging the API; in production the client uses Authorization Code + PKCE):

```
curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password \
  -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru \
  -d password='Admin123!' | python3 -m json.tool
```

Using the token:

```
TOKEN=$(curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru -d password='Admin123!' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["access_token"])')

curl -s http://localhost:8000/api/auth/me -H "Authorization: Bearer $TOKEN"
```

Moving from JWT to Keycloak

1. Bring up Keycloak, import the realm, verify JWKS.
2. Make sure the CRM users' e-mails match the e-mails in Keycloak: accounts are matched by that field.
3. Set `AUTH_MODE=keycloak` and restart the backend.
4. Verify sign-in: `GET /api/auth/me` with a Keycloak token.

The reverse move is to return `AUTH_MODE=jwt`. Both modes coexist, and user data does not change.

Limitations of the current implementation

- **State storage.** A local run uses `start-dev` with the embedded H2 database. A closed contour needs `start --optimized` with an external PostgreSQL and HTTPS enabled: in dev mode Keycloak is not built for load.
- **`sslRequired: none`** in the realm file allows HTTP: the demo stand is opened at `http://<vm-ip>:8080`, and with `external` Keycloak would reject such a sign-in. For an industrial contour `external` must be restored and TLS enabled everywhere.
- **Signing out.** Logout happens on the Keycloak side; the backend keeps no list of revoked tokens, so an access token lives until `exp`. Reducing `accessTokenLifespan` in the realm shortens that window.